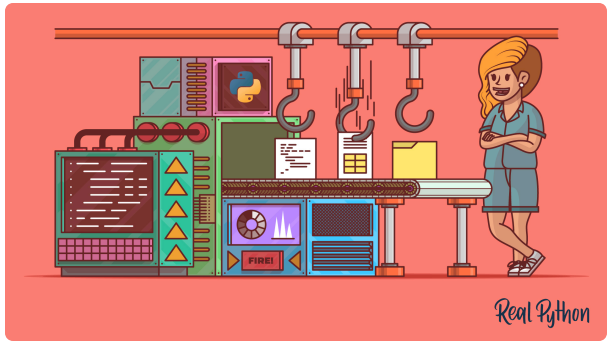


Real Python
Claude Code Hooks Cheat Sheet

Practice what you learn at realpython.com with hands-on coding exercises, quizzes, and guided learning paths.

Hook Anatomy

- A hook is a command that runs on an **event**, filtered by a **matcher**
- It fires every time, no matter what the model decides
- Omit `matcher` or use `"*"` to match every tool
- `"Bash"` matches one tool, `"Write|Edit"` several, `"^Notebook"` is a regex
- Hooks merge across files; commit `.claude/settings.json` to share them

Settings file	Scope
`~/.claude/settings.json`	All your projects
`.claude/settings.json`	This project, shareable
`.claude/settings.local.json`	This project, gitignored

Register a Hook

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "uv run .claude/hooks/guard_pip.py"
          }
        ]
      }
    ]
  }
}
```

Learn More on realpython.com/search:

`claude code hooks` · `claude code` · `CLAUDE.md`

Lifecycle Events

- Exit `2` can block `PreToolUse`, `UserPromptSubmit`, `Stop`, and `PreCompact`
- `PostToolUse` can't block; the tool already ran
- Tool hooks also fire inside subagents (`agent_id` is set)

Event	Fires
`SessionStart`	A session begins or resumes
`UserPromptSubmit`	You submit a prompt
`PreToolUse`	Before a tool call runs
`PostToolUse`	After a tool call succeeds
`Notification`	Claude Code needs your attention
`Stop`	Claude finishes a response
`PreCompact`	Before context compaction
`SessionEnd`	The session terminates

Learn More on realpython.com/search:

`claude code` · `agentic coding`

Read the Event

- The event arrives as JSON on standard input
- Always present: `hook_event_name`, `session_id`, `cwd`
- Tool events add `tool_name` and `tool_input`

Parse the Payload

```
import json
import sys

event = json.load(sys.stdin)
event["hook_event_name"]  # 'PreToolUse'
event["tool_name"]        # 'Bash'
event["tool_input"]       # {'command': 'pip install rich'}
```

Learn More on realpython.com/search:

`json` · `sys.stdin`

Exit-Code Contract

Exit code	Effect
`0`	Proceed; stdout may carry JSON
`2`	Block; stderr goes to Claude as feedback
Other	Non-blocking error; the action proceeds

- `exit 1` does **not** block. Only `2` does
- Steer, don't just deny: tell Claude what to run instead
- Stderr on exit `0` goes to the debug log only; Claude never sees it
- A hook that hits its timeout (600 s default) is discarded; the action proceeds

Block pip, Suggest uv

```
import json
import sys

event = json.load(sys.stdin)
cmd = event.get("tool_input", {}).get("command", "")
if cmd.startswith(("pip ", "pip3 ")):
    print("Use 'uv add' instead of pip.", file=sys.stderr)
    sys.exit(2)  # Blocked; Claude reads the message
sys.exit(0)  # Allowed
```

Test the Guard Without Claude

```
$ echo '{"tool_name": "Bash", "tool_input":
  {"command": "pip install rich"}}' \
  | uv run .claude/hooks/guard_pip.py; echo "exit $?"
Use 'uv add' instead of pip.
exit 2
```

Learn More on realpython.com/search:

`exit codes` · `uv`

Auto-Format Every Edit

- Match `"Write|Edit"` on `PostToolUse`
- Hooks inherit your shell `PATH`, not the project's virtual environment
- Run `ruff check --fix` before `ruff format`
- `PostToolUse` also sees `tool_response`, the tool's result
- Files rewritten by a `Bash` command don't trigger `Write|Edit` hooks

Put Ruff on Your PATH

```
$ uv tool install ruff
Installed 1 executable: ruff
```

Format the Changed File

```
import json
import subprocess
import sys
from pathlib import Path

event = json.load(sys.stdin)
path = event.get("tool_input", {}).get("file_path", "")
if path.endswith(".py") and Path(path).exists():
    subprocess.run(["ruff", "check", "--fix", path])
    subprocess.run(["ruff", "format", path])
```

Learn More on realpython.com/search:

`ruff` · `subprocess` · `pathlib`

Notify When Claude Stops

- Register under `"Stop"`, which fires after every response, not just the last

Desktop Notification

```
import json
import platform
import subprocess
import sys

json.load(sys.stdin)  # Consume the event
msg = "Claude just finished responding"
if platform.system() == "Darwin":
    note = f'display notification "{msg}"'
    cmd = ["osascript", "-e", note]
else:
    cmd = ["notify-send", "Claude Code", msg]
try:
    subprocess.run(cmd)
except FileNotFoundError:
    print(msg)  # No notifier installed? Print instead
```

Learn More on realpython.com/search:

`platform` · `subprocess`

Structured JSON Replies

- Exit `0` and print one JSON object for finer control
- `Stop` and `PostToolUse`: `{"decision": "block", "reason": "..."}`

Allow, Deny, or Ask on PreToolUse

```
import json

reply = {"hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": "ask",  # One of allow, deny, ask
    "permissionDecisionReason": "This edits prod config",
}}
print(json.dumps(reply))
```

Troubleshooting

- Not firing? Check the event name, the matcher, and the JSON commas
- `/hooks` lists what actually loaded and from which file

Trace a Hook in the Debug Log

```
$ claude --debug-file /tmp/hooks-debug.txt
$ grep -A2 "Hook" /tmp/hooks-debug.txt
[DEBUG] Hook PreToolUse:Bash (PreToolUse) error:
Use 'uv add' instead of pip.
[DEBUG] Hook denied tool use for Bash
```